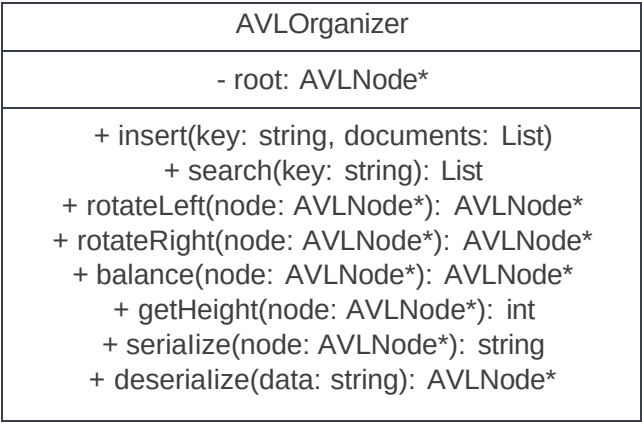
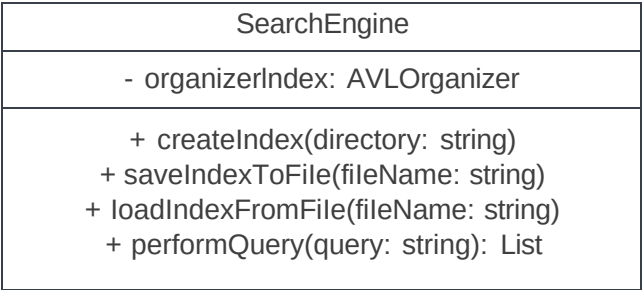


- **Search Engine (4/25 - estimated completion with *new additions*)**
 - Manages the searching of the search engine
 - Handles user queries, processes them, and retrieves relevant results from the data structures implemented:
 - Indexes: A hash map that stores the mapping between words and the web pages (including the provided database) where they appear. Each word or key word points to a list of document IDs from the provided file database.
 - ADDITION: Utilize multimaps for a better time complexity
 - When indexing, we can store the words as keys and the web pages as where the words are contained in
 - When using multiple web pages, multimaps are made to be more efficient for handling keys and values in multiple cases of where they are contained
 - Rank: Ranks search results based on relevance (think about how Google shows you the most relevant results first based on the words, especially the key words!) Deals with query and page rank.
 - ADDITION: Might utilize multimaps here as well for better time complexity
 - Same as from the indexes, but when dealing with multiple web pages, multimaps can handle efficient lookup with relevant values along with the web pages themselves
- **AVLOrganizer (4/20 - estimated completion with *corrections*)**
 - Organizes and manages the AVL trees containing different sets of data
 - Ensures efficient insertion, deletion, and retrieval functions (better memory management)
 - Data Structure:
 - Uses AVL trees to organize and store data such as words and persons
 - Will have to update from using red-black trees in the AVLTree class
- **AVLTree (4/20 - estimated completion with *new additions*)**
 - Provides methods for insertion, deletion, and searching elements in the tree
 - Data Structure:
 - Each node in the AVL tree contains data (key and documents), a pointer to the left and right child classes, and height for balancing the positions in tree
 - ADDITION: Utilize red-black trees for self-balancing
 - There will be a combination of AVLTree and red-black tree data structures as we can use the red-black trees

for handling individual nodes

- This will lead to faster insertion and deletion time complexities with red-black trees
- Also, the red-black trees can handle the rebalancing as the data structure self-balances itself, which limits the number of rotations in the levels of the tree, leading to a more efficient AVLTree structure when combined

- **AVLTreeOrganizer (4/20 - estimated completion with *corrections*)**

- A child class of the AVL tree for organizing documents in the list and the key words that are presented
- Handles the functions insert and search that are indexed by the search engine
 - Will have to update from using red-black trees in the AVLTree class

- **AVLTreeWords (4/20 - estimated completion with *corrections*)**

- A child class of the AVL tree for storing words especially key words extracted from the documents in the list
- Facilitates the indexing for the search engine through the functions insert and search
 - Will have to update from using red-black trees in the AVLTree class

- **AVLTreePerson (4/20 - estimated completion with *corrections*)**

- A child class of the AVL tree for storing and organizing information about individuals through insert and search functions that supplements the search engine indexing
 - Will have to update from using red-black trees in the AVLTree class